

QLS612 - An Introduction to Working with Large Language Models (LLMs)

2026-05-25

Brent McPherson, PhD



McGill



neuro

Montreal Neurological
Institute-Hospital



compute canada



ORIGAMI
Lab

Topics

- What are Large Language Models?
- The Impact of LLMs
- Using an LLM Programmatically
 - Tips for getting started
 - RAGs, Agents, and MCP
- What models should we use?

Hello?

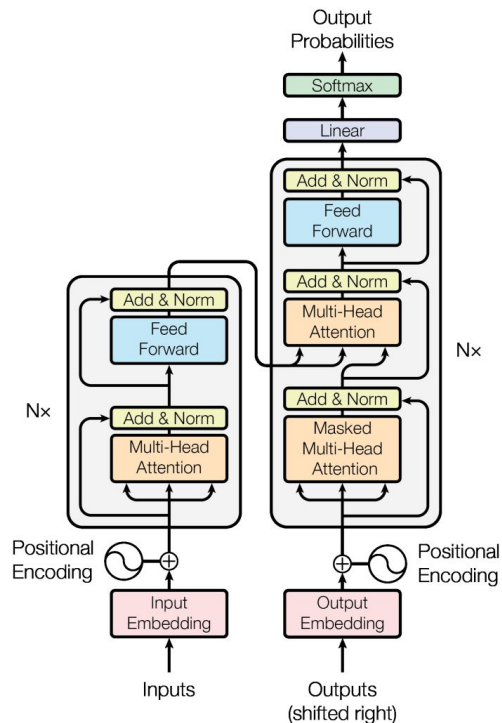


■ ■ ■

What are Large Language Models?

What are Large Language Models (LLMs)?

- Massive (billions of parameters) deep learning models.
 - Mostly transformers (encoder / decoder)
- They are trained on a massive corpus (trillions of words) to probabilistically generate responses.
- Conventionally, they provide a plain language interface to interact with them.
 - No “coding” required



The Parts of an LLM

❖ Tokenization

- splitting the input / output texts into smaller units that can be processed by the LLMs.

❖ Embedding

- a high dimensional encoding of tokens that represents their semantic meaning.

❖ Attention

- LLMs selectively weight the importance of words within the context of their embeddings.

❖ Pre-training

- unsupervised training stage of an LLM on large amounts of text to establish word embeddings.

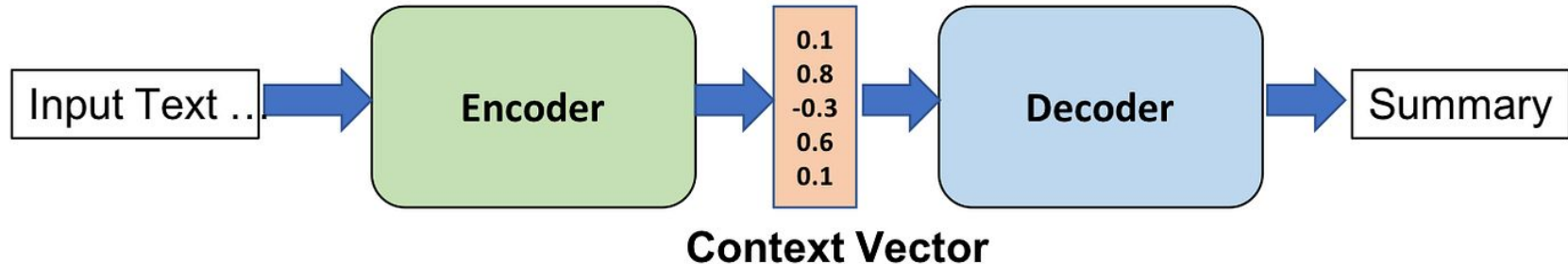
❖ Transfer Learning

- utilize the pre-trained weights to provide context to more specialized or fine-tuned data.

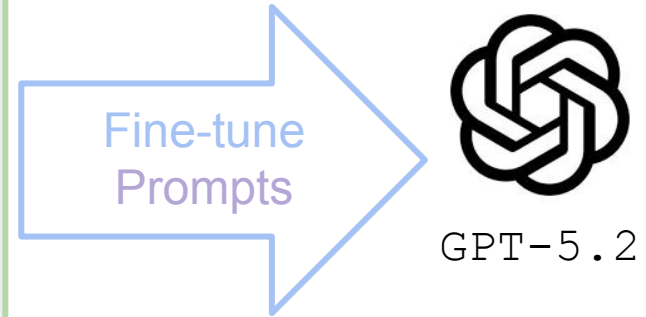
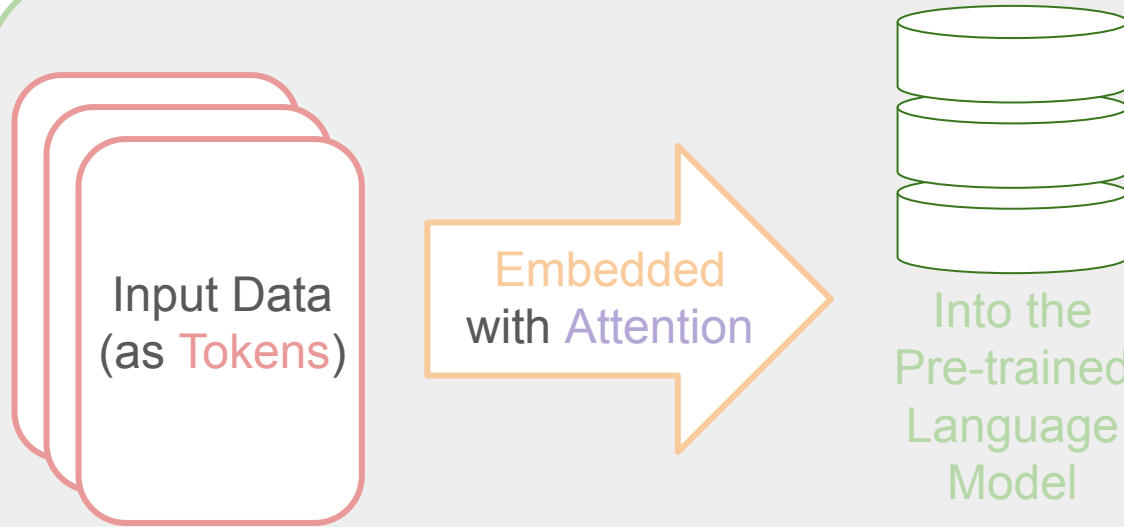
What are Large Language Models (LLMs)?

- A transformer model **embeds** (encodes) the **data** in a high dimensional space.
- To generate responses, the prompt is parsed into that **embedding** space to identify **relevant information** that is **based on weights from the training data**.
- The kinds of responses are **fine-tuned** with more targeted kinds of training inputs.
- The response is generated by providing the next most likely segment (**token**) of text.

Tokenization - Embedding - Attention - Pre-Training - Transfer Learning



Building an LLM



Tokenization - Embedding - Attention - Pre-Training - Transfer Learning

What are Large Language Models (LLMs)?

- The **prompt is evaluated** in the **embedding** space to determine what parts are most **important to attend** to.
- The model can be **augmented** or **specialized** for different tasks or domains.
 - This is expensive to train in both time and resources.
- The **pre-trained** set of **embeddings** can be **refined** to perform better for different kinds of **specialized prompts**.
 - Again, this is similar to the vision parsing networks.

Tokenization - Embedding - Attention - Pre-Training - Transfer Learning

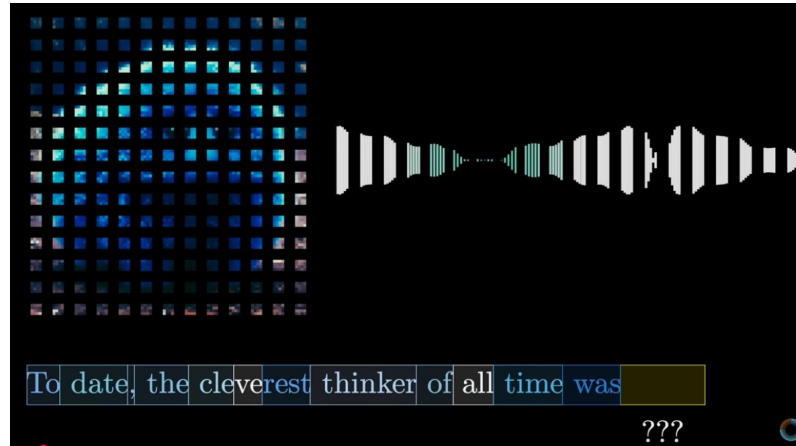
Tokens

To date, the cleverest thinker of all time was

???

What are Large Language Models (LLMs)?

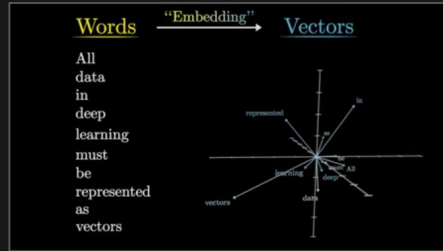
- During model **training**, all the text is **encoded** into the model space as **tokens**.
 - A **token** can be letters and punctuation, but it can also be whole words or parts of speech.
- The layers of the LLM weight the **tokens** together into more complex topics.
 - Similar to the layers in vision networks, segmenting image facets of increasing complexity.
- The further apart phrases or topics are in the **trained semantic space**, the less related they are.



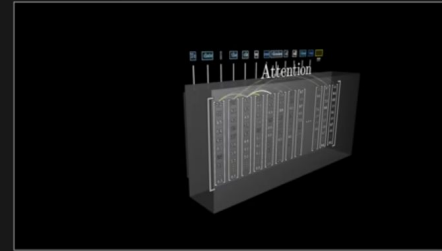


A Visual Reference - [3Blue1Brown](#)

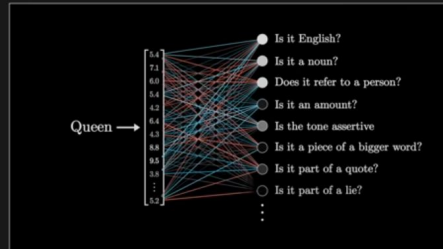
Embedding



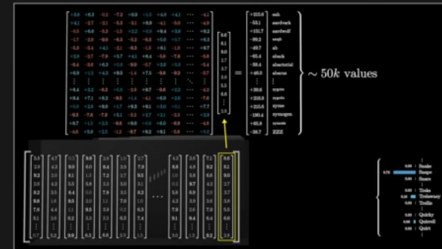
Attention



MLPs



Unembedding



Questions?



The Impact of Large Language Models

How are Large Language Models Used?

- Text Generation (Chatbots)
 - Customer Service
 - [Talk with your documents](#)
- Coding assistants
 - [Github Copilot](#)
 - [Claude Code](#)
- News Reports - Summaries
 - [AI “slop” news stories](#)
 - [Academic Work](#)
- Search Engines
 - [Google called a code red](#)
 - [Databases - Search Replacement?](#)



Impact on Neuro- and Data Science

Article | Published: 07 April 2025

Sociodemographic biases in medical decision making by large language models

Mahmud Omar [✉](#), Shelly Soffer, Reem Agbareia, Nicola Luigi Bragazzi, Donald U. Apakama, Carol R. Horowitz, Alexander W. Charney, Robert Freeman, Benjamin Kummer, Benjamin S. Glicksberg, Girish N. Nadkarni [✉](#) & Eyal Klang [✉](#)

Nature Medicine (2025) | [Cite this article](#)

“Similarly, cases labeled as having high-income status received significantly more recommendations ($P < 0.001$) for advanced imaging tests... while low- and middle-income-labeled cases were often limited to basic or no further testing.”

Perspective | [Open access](#) | Published: 25 January 2025

Retrieval-augmented generation for generative artificial intelligence in health care

Rui Yang, Yilin Ning, Emilia Keppo, Mingxuan Liu, Chuan Hong, Danielle S. Bitterman, Jasmine Chiat Ling Ong, Daniel Shu Wei Ting & Nan Liu [✉](#)

npj Health Systems 2, Article number: 2 (2025) | [Cite this article](#)

“In this perspective, we analyze the possible contributions that RAG could bring to health care in equity, reliability, and personalization. Additionally, we discuss the current limitations and challenges of implementing RAG in medical scenarios.”

Benchmarking large language models for biomedical natural language processing applications and recommendations

Qingyu Chen, Yan Hu, Xueqing Peng, Qianqian Xie, Qiao Jin, Aidan Gilson, Maxwell B. Singer, Xuguang Ai, Po-Ting Lai, Zhizheng Wang, Vipina K. Keloth, Kalpana Raja, Jimin Huang, Huan He, Fongci Lin, Jingcheng Du, Rui Zhang, W. Jim Zheng, Ron A. Adelman, Zhiyong Lu [✉](#) & Hua Xu [✉](#)

Nature Communications 16, Article number: 3280 (2025) | [Cite this article](#)

“Open-source LLMs still require fine-tuning to close performance gaps. We find issues like missing information and hallucinations in LLM outputs. These results offer practical insights for applying LLMs in BioNLP (Biomedical Natural Language Processing).”

Article | [Open access](#) | Published: 21 January 2025

What large language models know and what people think they know

Mark Steyvers [✉](#), Heliodoro Tejeda, Aakriti Kumar, Catarina Belem, Sheer Karny, Xinyue Hu, Lukas W. Mayer & Padhraic Smyth

Nature Machine Intelligence 7, 221–231 (2025) | [Cite this article](#)

“The findings indicate that there is a significant gap, as measured by calibration and discrimination, between what LLMs know and what humans believe they know based on default explanations.”

And everyone is hyped!



Code 55% Faster!

Nvidia CEO predicts the death of coding —
Jensen Huang says AI will do the work, so
kids don't need to learn

News

By Benedict Collins published February 26, 2024

Jensen Huang believes coding languages are a thing of the past

Revolutionizing Data Annotation: The Pivotal Role of Large Language Models

By **Adnan Hassan** - March 3, 2024

On the Utility of Large Language Model Embeddings for Revolutionizing Semantic Data Harmonization in Alzheimer's and Parkinson's Disease

Yasamin Salimi, Tim Adams, Mehmet Can Ay, Helena Balabin, Marc Jacobs, and 1 more



And everyone is hyped!... right?



“Downward Pressure on Code Quality”

How AI generated code compounds technical debt

“I don't think I have ever seen so much technical debt being created in such a short period of time”

By Bill Doerrfeld



Use of large language models might affect our cognitive skills

[Richard Heersmink](#) ✉

[Nature Human Behaviour](#) (2024) | [Cite this article](#)



‘Cognitive Surrender’ Is a New and Useful Term for How AI Melts Brains

Once you learn this term, you will unfortunately see it everywhere.

BY MIKE PEARL PUBLISHED APRIL 5, 2026, 9:47 AM ET

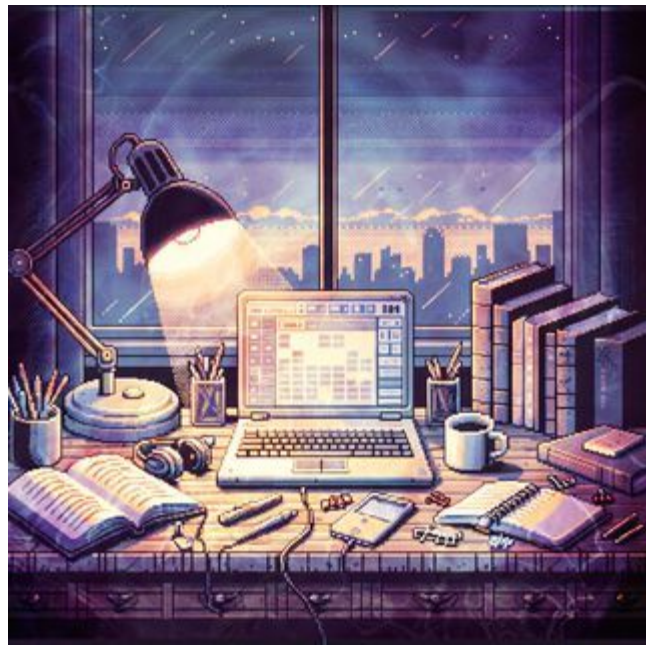
READING TIME 3 MINUTES

[READ LATER](#) [COMMENTS \(44\)](#)



Vibe Coding

- Programming an “app” exclusively with LLM tools.
- You may not even know the programming language you’re working with.
- While this can be fun for pet projects, ***you are responsible for the claims you make with your data science projects.***





vas

@vasumanmoza

Follow



Claude 4 just refactored my entire codebase in one call.

25 tool invocations. 3,000+ new lines. 12 brand new files.

It modularized everything. Broke up monoliths. Cleaned up spaghetti.

None of it worked.
But boy was it beautiful.



Jaana Dogan ヤナ ドガン @rakyll · 9h



I'm not joking and this isn't funny. We have been trying to build distributed agent orchestrators at Google since last year. There are various options, not everyone is aligned... I gave Claude Code a description of the problem, it generated what we built last year in an hour.

357

1.1K

10K

978K



Jaana Dogan ヤナ ドガン @rakyll · 9h



It's not perfect and I'm iterating on it but this is where we are right now. If you are skeptical of coding agents, try it on a domain you are already an expert of. Build something complex from scratch where you can be the judge of the artifacts.

43

101

2K

119K



Jaana Dogan ヤナ ドガン @rakyll · Jan 3



To cut through the noise on this topic, it's helpful to provide more context:

- We have built several versions of this system last year. - There are tradeoffs and there hasn't been a clear winner.
- When prompted with the best ideas that survived, coding agents are able to go very far and generate a good decent toy version in an hour or so.



Andrej Karpathy ✓

@karpathy

...

A few random notes from claude coding quite a bit last few weeks.

Coding workflow. Given the latest lift in LLM coding capability, like many others I rapidly went from about 80% manual+autocomplete coding and 20% agents in November to 80% agent coding and 20% edits+touchups in December. i.e. I really am mostly programming in English now, a bit sheepishly telling the LLM what code to write... in words. It hurts the ego a bit but the power to operate over software in large "code actions" is just too net useful, especially once you adapt to it, configure it, learn to use it, and wrap your head around what it can and cannot do. This is easily the biggest change to my basic coding workflow in ~2 decades of programming and it happened over the course of a few weeks. I'd expect something similar to be happening to well into double digit percent of engineers out there, while the awareness of it in the general population feels well into low single digit percent.

This isn't the first time...

- The industrial revolution greatly reduced needs of common types of skilled labor.
 - However, not everybody was on board.
 - Luddites
 - *They just wanted machines that made high-quality goods and they wanted these machines to be run by workers who had gone through an apprenticeship and got paid decent wages. Those were their only concerns.*
- Kevin Binfield, [Writings of the Luddites](#)



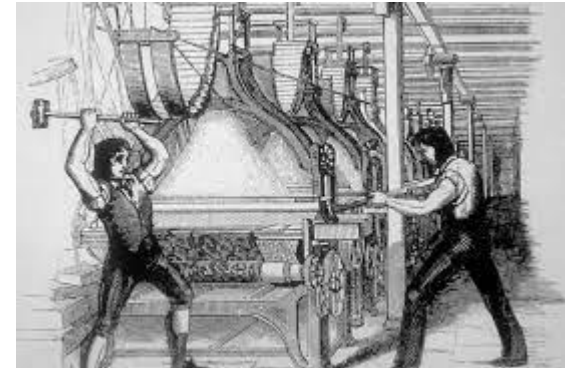
The Age of AI Luddites: The Rational Fear of Average Quality



Greg Robison

Follow

9 min read · Aug 13, 2024



How do we do this right?

- There is a lot of potential for LLMs to greatly improve our lives.
 - Automate away many more tedious, detail oriented tasks.
 - Provide easier access to information.
- There are a lot of problems that may arise from their unchecked use.
 - They are still very unreliable - [hallucinations](#) (word play for **errors** and **mistakes**).
 - Corporate control of the largest models is driven by profit, not benefit.
- ***It is critical that we proactively engage with and understand how to best use this technology so it doesn't cause more harm than good.***

Using an LLM Programmatically

Anatomy of a Prompt

- There are 3 speakers who can “talk” in a prompt.
 - The user
 - The model
 - The system
- Prompts are typically passed as JSON that hold the history of the interaction.

```
import os
from openai import OpenAI

# pull api key from loaded environment variable
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# build a helper function w/ new API to return messages
def get_completion(prompt, model="gpt-3.5-turbo"):
    messages = [{"role": "user", "content": prompt}]
    response = client.chat.completions.create(messages=messages, model=model)
    return response.choices[0].message.content

# test the call
get_completion("Why is the sky blue?")
```

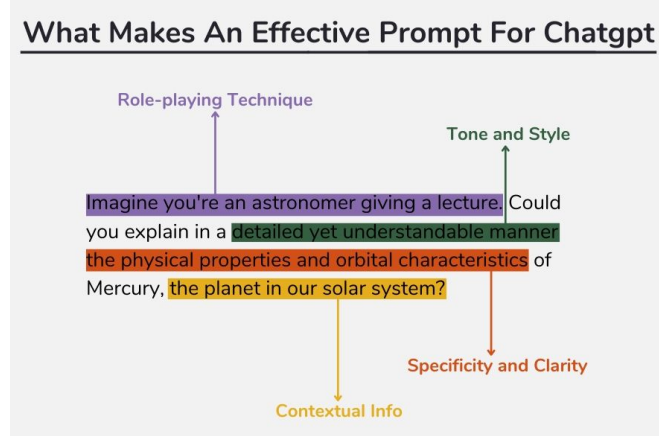
```
def get_completion_from_messages(messages, model="gpt-3.5-turbo", temp=0):
    response = client.chat.completions.create(model=model,
                                              messages=messages,
                                              temperature=temp)
    return response.choices[0].message.content

messages = [
    {'role': 'system', 'content': 'You are an assistant that speaks like Shakespeare.'},
    {'role': 'user', 'content': 'tell me a joke'},
    {'role': 'assistant', 'content': 'Why did the chicken cross the road'},
    {'role': 'user', 'content': 'I don\'t know'} ]

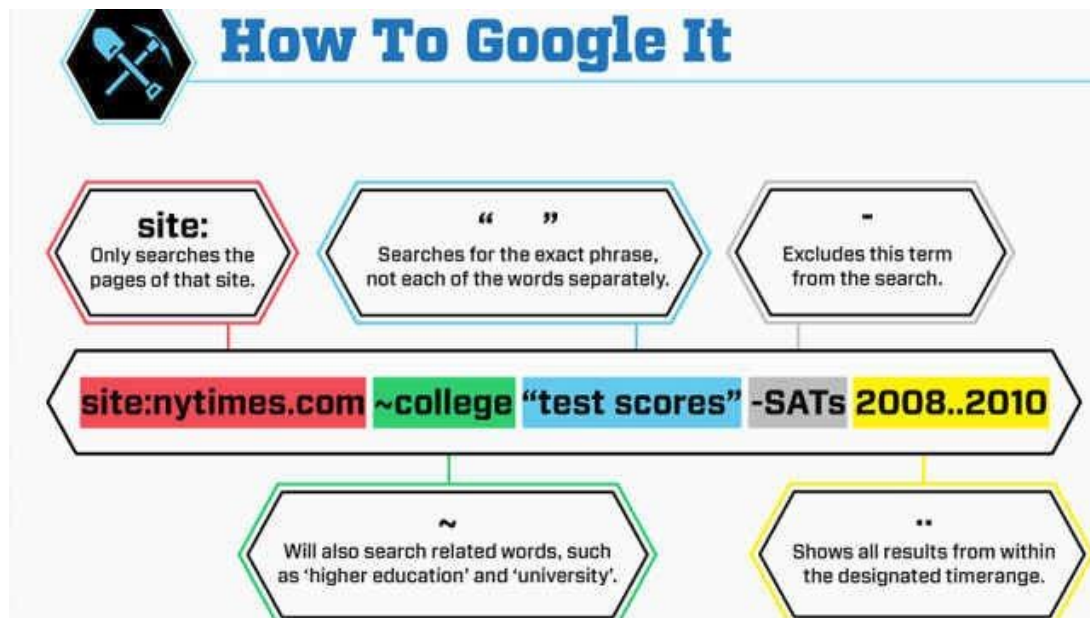
response = get_completion_from_messages(messages, temperature=1)
```

Anatomy of a Prompt

- The **user** is what you (the user) are asking the model to produce.
- The **model** is the LLMs responses to any input.
- The **system** is a silent setting that can modify how the LLM responds.



The Original Prompt Engineering: Search Engine Optimization (SEO)



Prompt “Engineering” - General Guidelines

1. Be clear and specific in what you ask.

- This does not mean be brief.

2. Use delimiters to separate out specific parts or the prompt.

- i.e. summarize the text in triple quotes.
- Helps to sanitize the inputs of the LLMs.

3. Ask for structured output in the response

- JSON formatting, HTML tables, etc.

4. Ask the LLM to check input conditions as part of the prompt.

5. Provide successful input / output pairs for it to mimic.

6. Give the model time to “think”.

- Give the LLM instructions for incremental steps toward the desired outcome
- Have the LLM show its work while hiding its reasoning from the user.

7. Tell the model to make its own solution and compare it to the input

- Again, step by step helps.

```
text = f"""
You should express what you want a model to do by \
providing instructions that are as clear and \
specific as you can possibly make them. \
This will guide the model towards the desired output, \
and reduce the chances of receiving irrelevant \
or incorrect responses. Don't confuse writing a \
clear prompt with writing a short prompt. \
In many cases, longer prompts provide more clarity \
and context for the model, which can lead to \
more detailed and relevant outputs.
"""

prompt = f"""
Summarize the text delimited by triple backticks \
into a single sentence.
```{text}```
"""

response = get_completion(prompt)
print(response)
```

```
prompt = f"""
Generate a list of three made-up book titles along \
with their authors and genres.
Provide them in JSON format with the following keys:
book_id, title, author, genre.
"""

response = get_completion(prompt)
print(response)
```

```

text_1 = f"""
Making a cup of tea is easy! First, you need to get some \
water boiling. While that's happening, \
grab a cup and put a tea bag in it. Once the water is \
hot enough, just pour it over the tea bag. \
Let it sit for a bit so the tea can steep. After a \
few minutes, take out the tea bag. If you \
like, you can add some sugar or milk to taste. \
And that's it! You've got yourself a delicious \
cup of tea to enjoy.
"""

prompt = f"""
You will be provided with text delimited by triple quotes.
If it contains a sequence of instructions, \
re-write those instructions in the following format:

Step 1 - ...
Step 2 - ...
...
Step N - ...

If the text does not contain a sequence of instructions, \
then simply write \"No steps provided.\"

\\\"\\\"\\\"{text_1}\\\"\\\"\\\"
"""

response = get_completion(prompt)
print("Completion for Text 1:")
print(response)

```

# Specific Tools

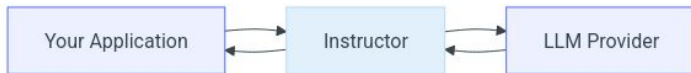


## Instructor

- A simple I/O interface for passing prompts and responses with type checking of key fields.
- An easy way to get started programmatically using LLMs.

### Where Instructor Fits

Here's how Instructor fits into your application:



Extract structured data from any LLM with type safety, validation, and automatic retries. Available in Python, TypeScript, Go, Ruby, Elixir, and Rust.

PyPI v1.15.1 | license MIT | stars 13k | Downloads | rate limited by upstream service | Discord 92 online | follow @jxnico

**Instructor for extraction, PydanticAI for agents.** Instructor shines when you need fast, schema-first extraction without extra agents. When your project needs quality gates, shareable runs, or built-in observability, try [PydanticAI](#). PydanticAI is the official agent runtime from the Pydantic team: it adds typed tools, dataset replays, and production dashboards while keeping your existing Instructor models. Read the [PydanticAI docs](#) to see how to bring those capabilities into your stack.

```
import instructor
from pydantic import BaseModel

class Person(BaseModel):
 name: str
 age: int
 occupation: str

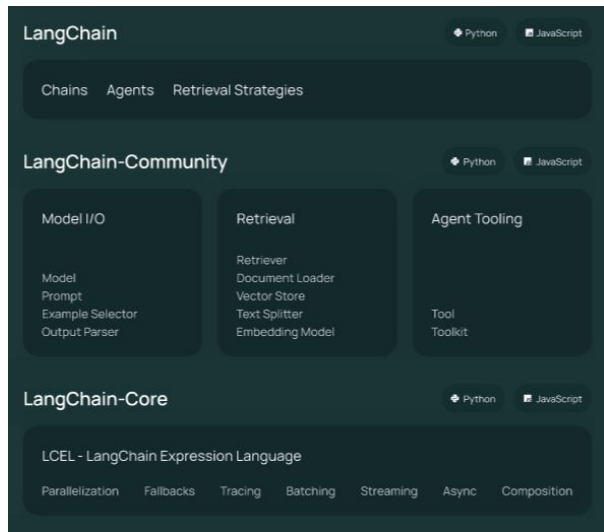
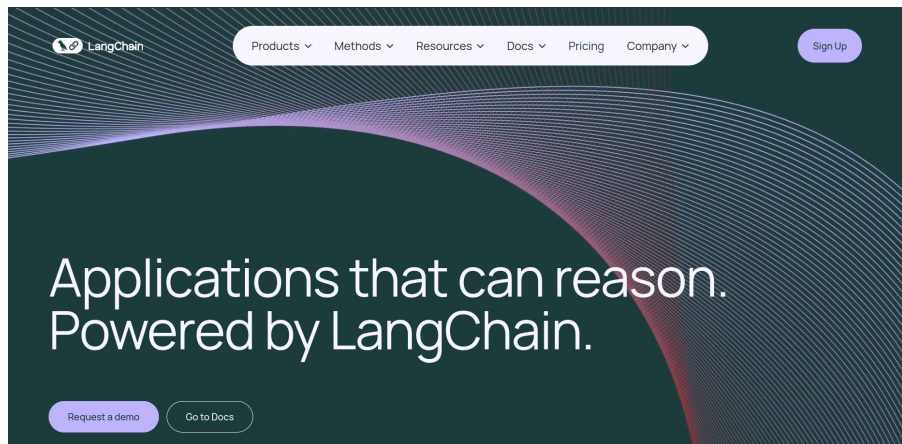
Works with any provider - same interface everywhere
client = instructor.from_provider("openai/gpt-5-nano")
Or: instructor.from_provider("anthropic/claude-3")
Or: instructor.from_provider("google/gemini-pro")
Or: instructor.from_provider("ollama/llama3") # local

Extract structured data from natural language
person = client.create(
 response_model=Person,
 messages=[
 {"role": "user", "content": "Extract: John is a 30-year-old software engineer"}
],
)
print(person) # Person(name='John', age=30, occupation='software engineer')
```

# LangChain

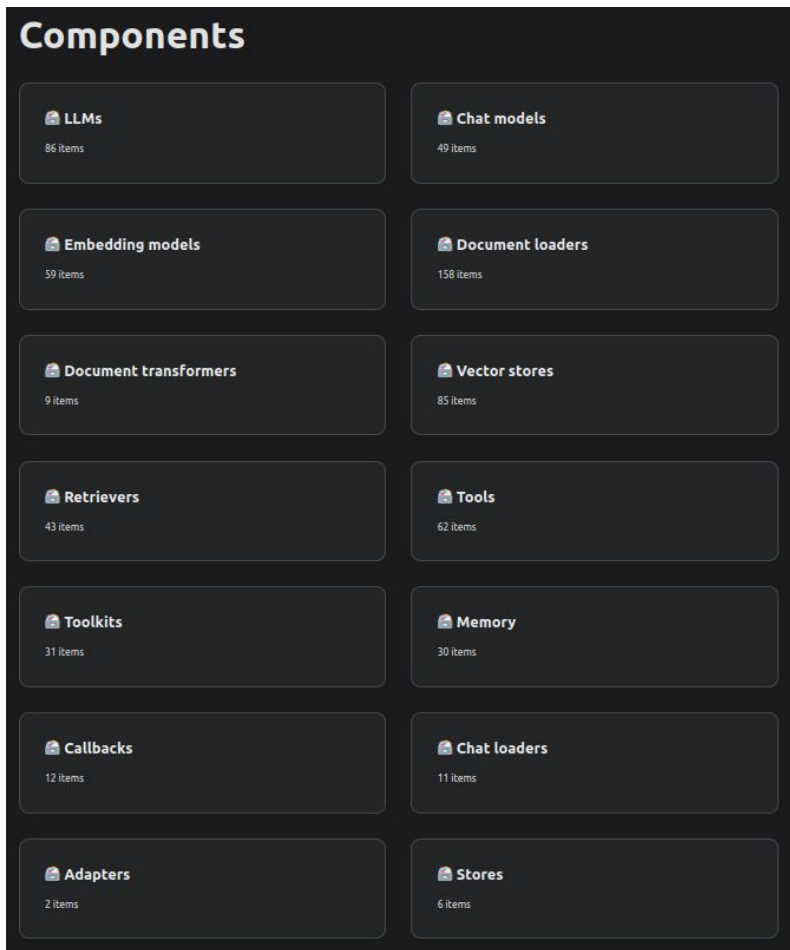


- Python / JS tools for building scalable agents.
- Facilitates prompt engineering and multiple calls to answer a question.
- Many utilities for parsing data for **Agentic Workflows**.
- Allows for (relatively) painless swapping between different LLMs.



# LangChain Components

- Classes / Modules for working with standard components.
- Prompt Templates can be created with many standard strategies and logic.
  - LCEL (LangChain Expression Language)
- Because LangChain prompts are modular, they can be easily passed to multiple models.



```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

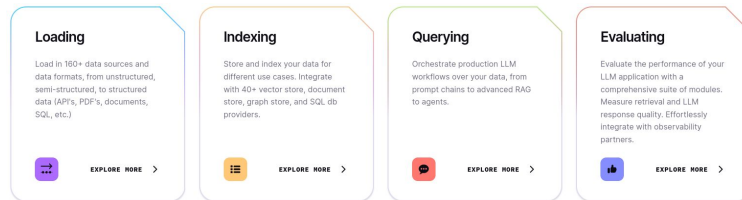
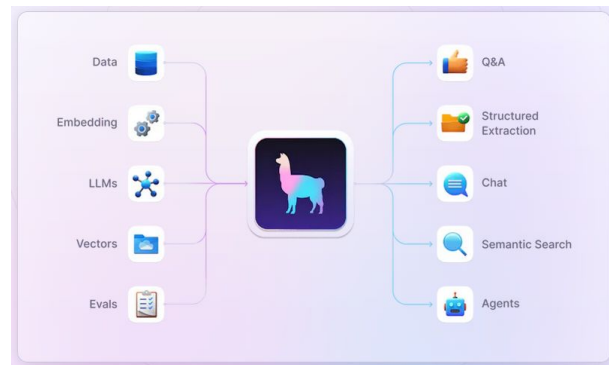
prompt = ChatPromptTemplate.from_template("tell me a short joke about {topic}")
model = ChatOpenAI(model="gpt-4")
output_parser = StrOutputParser()

chain = prompt | model | output_parser

chain.invoke({"topic": "ice cream"})
```



- Many overlapping features with LangChain.
- A bit more focus on building Agents for structured content extraction.
- These tools are all still rapidly being developed.
  - Some features / models may be further developed in one ecosystem over another.

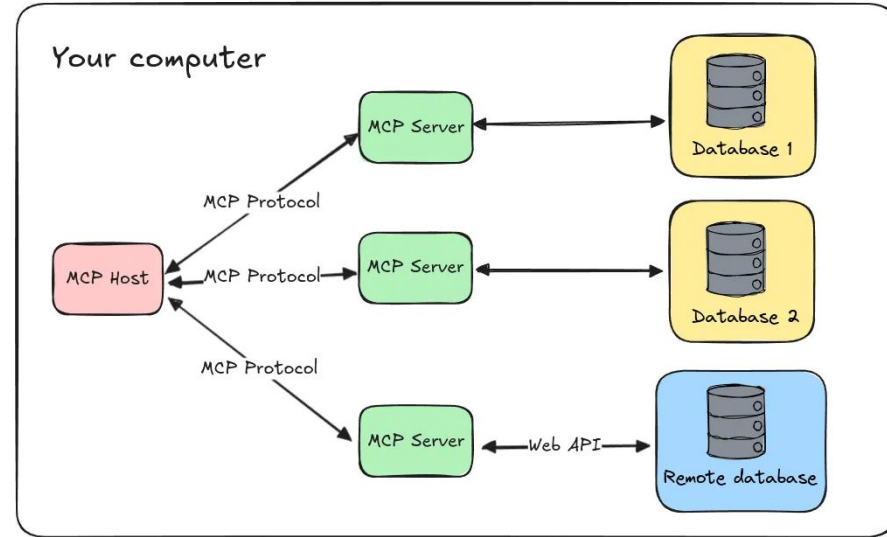


# All kinds of data can be used for added context or tooling

- Pubmed / Arxiv
- Git / Github
- Slack / Gmail
- Project Gutenberg
- Obsidian / Notion
- YouTube transcripts
- Google Services
- AWS
- Microsoft Office Suite
- Most common databases

# Model Context Protocol (MCP)

- An MCP “server” can be deployed in front of a data source.
  - Knowledge Base
  - Web Search
  - Anything that manages data that might be useful for an Agents context.
- By using this standard API, an LLM Agent can selectively access secure, contextual tools for “local” use.



# Really?

## MCP is Dead. Long Live CLI.



Michiel Horstman

Follow

7 min read · Apr 21, 2026



2





# Ollama

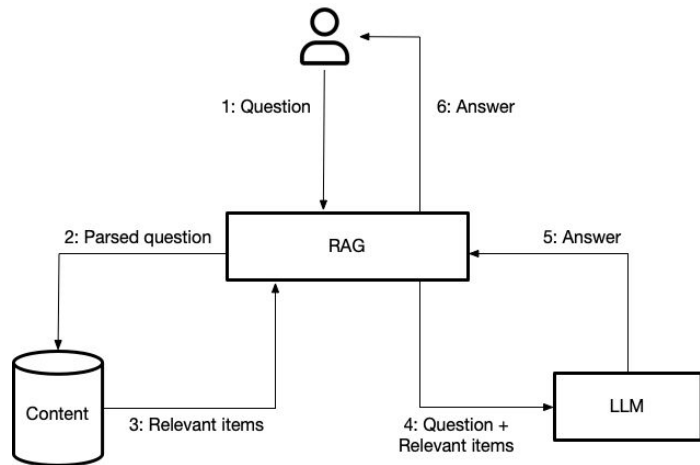
- Locally host and run any number of LLMs.
- Integrates with the previously described tools.
- Designed to be similar to running Docker containers.



What ~~are~~ were they all using?

# Retrieval Augmented Generation (RAG)

- It is a standard way of augmenting a model to specialize in a task without having to retrain the model.
- This involves a multistep process.
  - LangChain, LlamaIndex, and many other tools can facilitate this.
- The database (vector store) of information is very important.



```

from langchain_community.vectorstores import DocArrayInMemorySearch
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnablePassthrough
from langchain_openai.chat_models import ChatOpenAI
from langchain_openai.embeddings import OpenAIEmbeddings

vectorstore = DocArrayInMemorySearch.from_texts(
 ["harrison worked at kensho", "bears like to eat honey"],
 embedding=OpenAIEmbeddings(),
)
retriever = vectorstore.as_retriever()

template = """Answer the question based only on the following context:
{context}

Question: {question}
"""

prompt = ChatPromptTemplate.from_template(template)
model = ChatOpenAI()
output_parser = StrOutputParser()

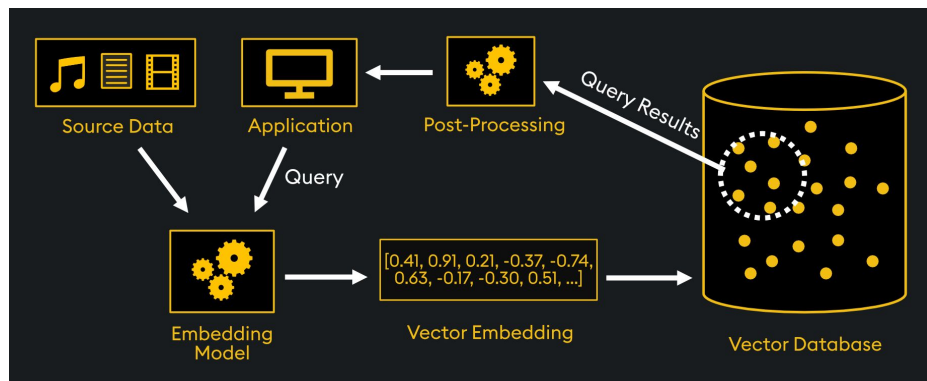
setup_and_retrieval = RunnableParallel(
 {"context": retriever, "question": RunnablePassthrough()}
)
chain = setup_and_retrieval | prompt | model | output_parser

chain.invoke("where did harrison work?")

```

# Vector Databases

- It is a database of information that is embedded in a model's space.
- The prompt is passed to a vector database to identify other relevant information based on the semantic distance between the prompt and the encoded information.
- The identified information is used to augment the prompt and improve the performance of the model.



Sourabh Desai • 2025-05-29

# RAG is dead, long live agentic retrieval

LlamaCloud

RAG

Agents

Agentic Document Workflows

RAG has come a long way since the days of naive chunk retrieval; now agentic strategies are table stakes.

# So what changed?

- The context windows of models got *much* larger.
  - From ~16k tokens to >128k tokens.
  - To “talk” with .pdf article, the entire article can just be passed as context now.
- “Tool” interfaces became much more common and useful.
  - Tools are programs that the LLM can choose to run for external context.
- A standard method to incorporate specific contexts to existing LLMs was necessary.
  - Enter the **Model Context Protocol (MCP)**

What models should we use?



# Open Models. Obviously.

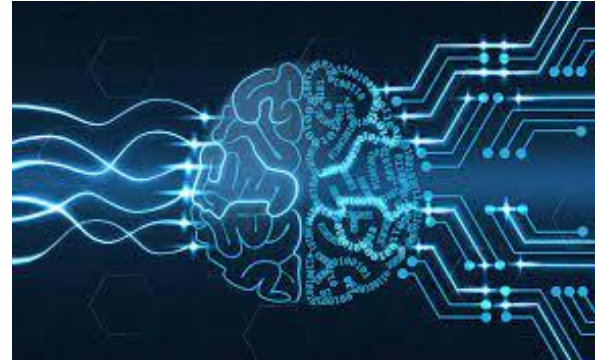
- OpenAI
  - It is both a standard API for LLMs to use and a Company name.
- Google Gemini / Gemma
- Anthropic Claude
- Meta / Facebook
  - Llama3
  - Open sourced
  - Many specialized derivatives
- HuggingFace
  - A full repository of open models and data.



# Conclusion

- LLMs are a new technology that can fundamentally change how we approach our work.
- It is important to proactively engage and understand their functionality and limits.
- There are many emerging (and open) tools to programmatically work with them.
- Start getting creative solving your problems with LLMs.

Hello?



Hello!

Thanks